

Guia de Desenvolvimento: Cofre Digital - Documentos Acadêmicos

Este documento serve como o guia técnico direto e estruturado para a equipe de desenvolvimento do Trabalho Prático 1, abordando especificamente o **Tema 2: Documentos Acadêmicos**. O projeto consiste em um Cofre Digital de Arquivos utilizando Python e FastAPI.

1. Tecnologias e Arquitetura Obrigatórias

- **Linguagem:** Python
- **Framework Web:** FastAPI
- **Modelagem e Validação:** Pydantic
- **Persistência de Dados (CRÍTICO):** Estritamente baseada em arquivos (JSON para metadados). **É expressamente proibido o uso de bancos de dados relacionais ou NoSQL para substituir o mecanismo de arquivos.**
- **Formatos Utilizados:** JSON, XML, CSV, YAML / .properties (configuração), binários e logs.

2. Estrutura de Diretórios (Substituto de Migrations)

Como não utilizamos bancos de dados, não há "migrations" SQL. O **setup** do sistema consiste em inicializar ou garantir a existência da seguinte estrutura de diretórios e arquivos de persistência:

```
C/C++
/storage
  /documentos/          # Armazenamento dos arquivos físicos
  (binários originais)
  /metadata/
    documentos.json     # Persistência principal (lista de
    dicionários JSON)
  /logs/
    sistema.log         # Arquivo de logging gerado pela API nativa
do Python
  /backups/             # Arquivos .zip gerados pelos endpoints de
  backup
  /export/              # Arquivos CSV e XML gerados
  config.yaml           # Arquivo principal de configuração externa
  da aplicação
```

3. Entidades (Modelos Pydantic)

A entidade principal unifica os metadados gerais exigidos pelo sistema e os metadados

específicos do domínio de **Documentos Acadêmicos**.

Python

```
from pydantic import BaseModel
from datetime import datetime
from typing import Optional

class DocumentoAcademico(BaseModel):
    # Metadados Gerais (Comuns a todos os temas)
    id: int
    nome_original: str
    nome_armazenado: str
    extensao: str
    tipo_mime: str
    tamanho: int
    categoria: str
    descricao: Optional[str] = None
    data_upload: datetime
    sha256: str

    # Metadados Específicos (Tema 2 - Documentos Acadêmicos)
    aluno: str
    matricula: str
    curso: str
    semestre: str
    tipo_documento: str
```

4. Endpoints da API (Funcionalidades F1 a F16)

Implemente estritamente as rotas abaixo, mapeando os requisitos solicitados:

Func.	Método	Endpoint	Descrição Resumida
F1	POST	/documentos	Upload de arquivo. Calcula SHA-256 , salva arquivo físico, gera metadados e adiciona ao documentos.json . Evita sobrescrita.
F2	GET	/documentos	Lista todos os documentos lendo diretamente arquivo de metadados JSON .
F3	GET	/documentos/{id}	Consulta metadados de um documento específico pelo identificador. Retorna 404 se existir.
F4	GET	/documentos/{id}/download	Baixa o arquivo binário original sem alteração de conteúdo.

Func.	Método	Endpoint	Descrição Resumida
F5	PUT	/documentos/{id}	Atualiza os metadados no arquivo JSON (categoria, descrição e campos acadêmicos).
F6	DELETE	/documentos/{id}	Remove o registro do JSON e exclui o arquivo físico correspondente.
F7	GET	/documentos/pesquisar	Filtra documentos. Requer 1 atributo geral (ex: extensao=pdf) e 2 específicos (ex: aluno=Maria, curso=Sistemas).
F8	GET	/documentos/estatisticas	Retorna total de documentos, espaço ocupado, agrupamento por extensão/categoria e estatísticas de curso/semestre.
F9	GET	/documentos/{id}/integridade	Recalcula o SHA-256 do arquivo físico e compara com o valor guardado no JSON .
F10	GET	/integridade	Verificação em lote: qtd verificados, qtd íntegros, qtd alterados, arquivos não localizados.
F13	GET	/exportar/csv	Gera e retorna um arquivo CSV com o catálogo completo, incluindo colunas específicas do tema acadêmico.
F14	POST	/backup	Gera um arquivo ZIP compactado com os dados e salva em storage/backups.
F15	GET	/backups	Lista os arquivos de backup disponíveis com nome e tamanho.

5. Requisito Específico do Tema 2 (F16)

Descrição: O sistema deve permitir consultar documentos por **aluno** ou **semestre** e gerar uma exportação em formato **XML** contendo os documentos selecionados.

- **Endpoint Específico:** GET
/documentos/exportar/xml?aluno={nome_aluno}&semestre={semestre_codigo}
- **Comportamento:** A rota deve filtrar os dados em memória (lidos do **JSON**), construir a estrutura em tags **XML** representando os documentos selecionados e retornar o arquivo XML para download (Response MIME Type: application/xml).

6. Arquivos e Configurações Adicionais

- **F11 - Log (sistema.log):** Registrar inicialização, uploads, downloads, consultas, alterações e, principalmente, falhas de integridade ou tentativa de acesso a arquivos inexistentes. Usar os níveis **INFO**, **WARNING** e **ERROR**.

- **F12 - Configuração (config.yaml):** Centralize variáveis como tamanho máximo de upload, caminho das pastas, nível de log e algoritmo de hash (**SHA-256**). O sistema **DEVE** ler este arquivo no boot.
- **F17 - Tratamento de Erros:** Retornar HTTP **400** (requisição inválida), **404** (não encontrado), **409** (conflito), **422** (dados inválidos) ou **500** para todas as anomalias, como JSON quebrado ou falhas de disco.

Dados Iniciais (Setup para Apresentação): O sistema deverá ser iniciado com pelo menos **15 documentos** já cadastrados no JSON e salvos na pasta. É mandatório contemplar pelo menos **4 extensões** (ex: PDF, DOCX, JPG, TXT) e metadados temáticos coerentes com documentos acadêmicos (matrículas, histórico, etc).